

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №2

Системы линейных алгебраических уравнений

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Прямой ход	2
2.1.2	Обратный ход	3
2.2	Блок-схема численного метода	3
2.3	Листинг численного метода	5
2.3.1	Метод Гаусса	5
2.3.2	Программный интерфейс	7
2.4	Результаты работы программы	18
2.4.1	Внешний ввод данных	18
2.4.2	Случайные матрицы	20
3	Заключение	24

1 Постановка задач

Дана система линейных алгебраических уравнений вида $Ax = f$, где A — матрица коэффициентов размерности $n \times n$, а f — вектор свободных членов.

В варианте №1 решение системы осуществляется *методом Гаусса*, а оценка корректности — через *расчёт вектора невязок*.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка программного модуля для реализации алгоритма Гаусса.
2. Реализация проверки системы на совместность и определенность.
3. Написание алгоритма вычисления вектора невязок r для оценки погрешности полученного результата.
4. Вычисление вектора решений x с вектором невязок r для данной СЛАУ.

2 Выполнение работы

2.1 Описание метода

2.1.1 Прямой ход

Рассмотрим общий вид СЛАУ размера n в матричном виде:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} \iff A\vec{x} = \vec{f}$$

Метод Гаусса предлагает привести матрицу коэффициентов к *треугольному виду*. Для определённости будем рассматривать *верхнетреугольные матрицы*, то есть матрицы (u_{ij}) , для которых верно:

$$i > j \implies u_{ij} = 0$$

Преобразование квадратной матрицы к верхнетреугольной возможно, если система имеет единственное решение. Покажем это конструктивным способом.

Пусть $a_{11} \neq 0$. Подействуем слева на равенство гомоморфизмом вида:

$$M_1 = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\frac{a_{21}}{a_{11}} & 1 & 0 & \cdots & 0 \\ -\frac{a_{31}}{a_{11}} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_{n1}}{a_{11}} & 0 & 0 & \cdots & 1 \end{pmatrix}$$

Это автоморфизм, поскольку $\det(M_1) \neq 0$. То есть шаг равносильен на $\mathbb{R}^n$.

Оператор M_1 описывает прибавление к i -й строке первой строки, умноженной на коэффициент $-a_{i1}/a_{11}$. В результате получим систему вида:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(1)} & \cdots & a_{nn}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2^{(1)} \\ \vdots \\ f_n^{(1)} \end{pmatrix}$$

Пусть $a_{11} = 0$. Найдём среди a_{i1} ($i > 1$) ненулевой элемент на j -ой строке. Подействуем слева на равенство гомоморфизмом вида:

$$P_{1j} = \begin{pmatrix} 0 & 0 & \cdots & 1 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 1 & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix},$$

где единица в первой строке стоит на j -м месте, а единица в j -й строке — на первом месте (*остальные диагональные элементы равны 1*).

Поскольку $\det P_{1j} \neq 0$, это преобразование также автоморфизм, что гарантирует сохранение множества решений системы.

Оператор P_{1j} описывает перестановку первой и j -ой строки местами. Так мы свели задачу к случаю $\tilde{a}_{11} \neq 0$.

Если весь первый столбец оказался нулевым, то базисный вектор $e_1 = (1, 0, \dots, 0)^T$ принадлежит ядру оператора A . Это делает отображение неинъективным, а его матрицу — вырожденной. То есть решения у системы либо нет, либо их бесконечно много.

Далее рассмотрим сужение полученной композиции операторов на подпространство $V_{n-1} \subset \mathbb{R}^n$, которое является прямым дополнением к $\text{span}\{e_1\}$. Это соответствует переходу к работе со следующей системой:

$$\begin{pmatrix} a_{22}^{(1)} & a_{23}^{(1)} & \cdots & a_{2n}^{(1)} \\ a_{32}^{(1)} & a_{33}^{(1)} & \cdots & a_{3n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n2}^{(1)} & a_{n3}^{(1)} & \cdots & a_{nn}^{(1)} \end{pmatrix} \begin{pmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_2^{(1)} \\ f_3^{(1)} \\ \vdots \\ f_n^{(1)} \end{pmatrix}$$

Этот процесс повторяется итерационно для $k = 2, 3, \dots, n-1$. На каждом шаге мы выбираем ведущий элемент $a_{kk}^{(k-1)}$, при необходимости применяя автоморфизм перестановки строк, и конструируем соответствующий оператор M_k . В результате последовательного применения композиции автоморфизмов исходная матрица A преобразуется в верхнетреугольную матрицу U .

2.1.2 Обратный ход

Рассмотрим общий вид СЛАУ размера n в матричном виде с верхнетреугольной матрицей U , полученной после завершения прямого хода:

$$\begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \tilde{f}_1 \\ \tilde{f}_2 \\ \vdots \\ \tilde{f}_n \end{pmatrix}$$

Если все диагональные элементы $u_{ii} \neq 0$, система имеет единственное решение. Метод Гаусса предлагает выразить компоненты $\vec{x}$ по следующей рекуррентной формуле:

$$x_i = \frac{1}{u_{ii}} \left(\tilde{f}_i - \sum_{j=i+1}^n u_{ij} x_j \right)$$

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- n — размерность системы; является целым числом.
- A — расширенная матрица системы; является двумерным массивом вещественных чисел двойной точности.

На выходе метода ожидается вектор над вещественными числами двойной точности — решение системы, которое совмещено с таким же вектором невязки для данного решения.

В случае, если система не имеет единственного решения, метод должен выбросить соответствующую ошибку.

Далее приводятся блок-схемы, реализующие численный метод.

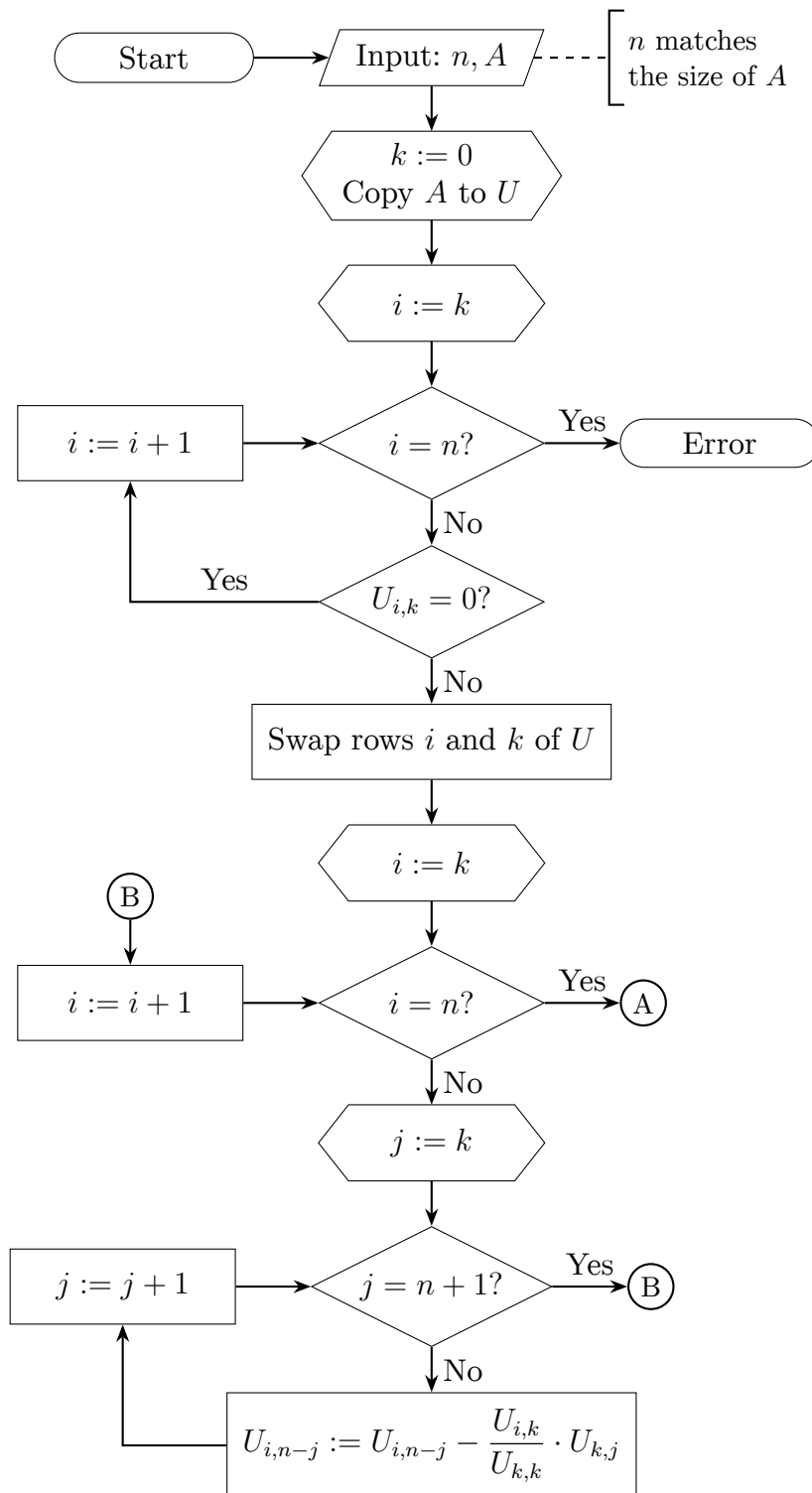


Схема 1 — Прямой ход метода Гаусса.

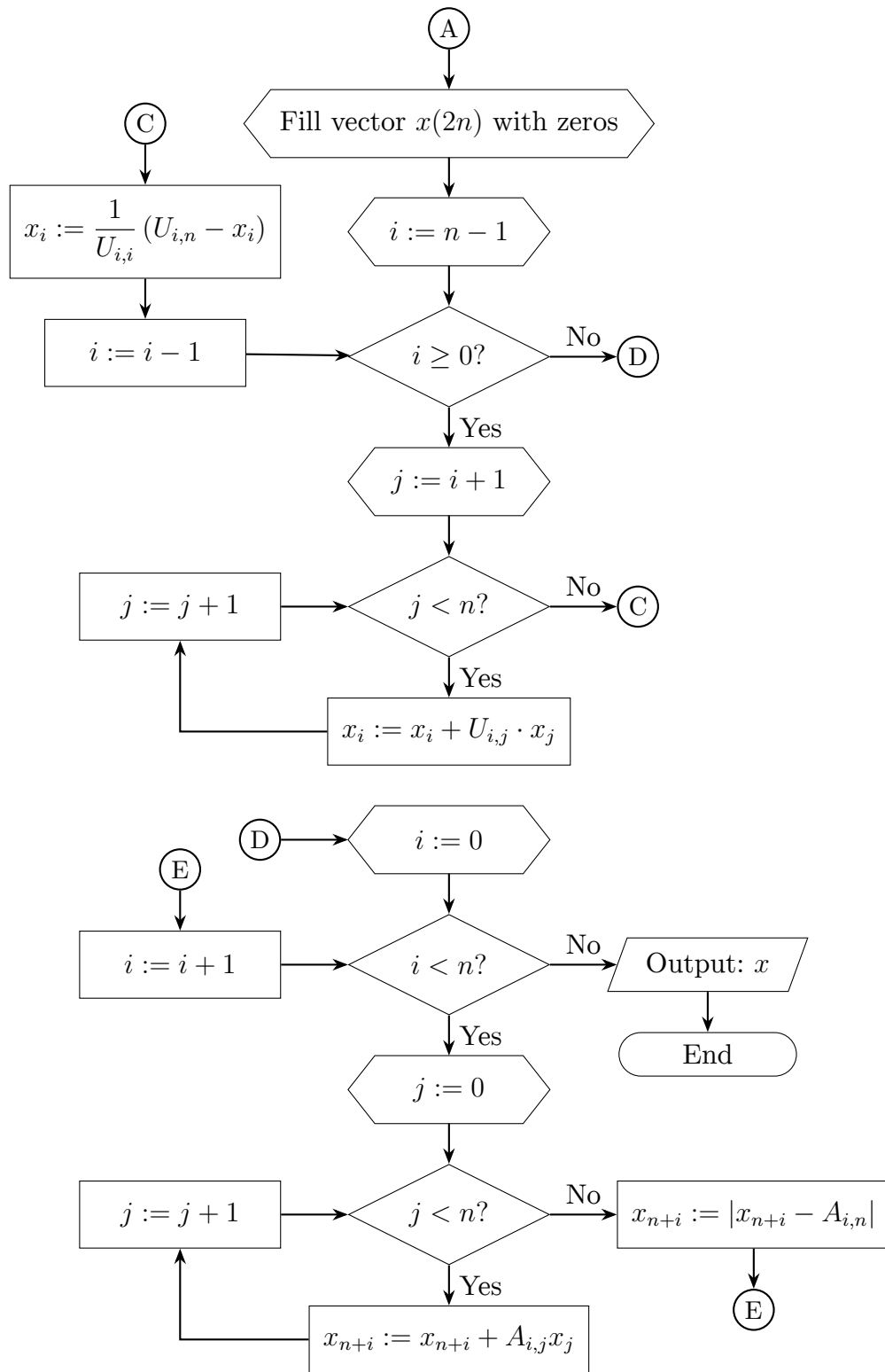


Схема 2 — Обратный ход метода Гаусса.

2.3 Листинг численного метода

2.3.1 Метод Гаусса

Для реализации блок-схем воспользуемся языком программирования C++.

```

1 bool triangulateMatrix(const int n, vector<vector<double>>&
   matrix) {
2     bool isSingular = false;
3     for (size_t i = 0; i < n - 1; i++) {
4         size_t idx = n;
5         for (size_t j = i; j < n; j++) {
6             if (matrix[j][i] != 0) {
7                 idx = j;
8                 break;
9             }
10        }
11        if (idx == n) {
12            isSingular = true;
13            continue;
14        }
15        // apply P operator
16        if (idx != i) {
17            vector<double> temp_row = matrix[idx];
18            matrix[idx] = matrix[i];
19            matrix[i] = temp_row;
20        }
21        // apply M operator
22        for (size_t j = i + 1; j < n; j++) {
23            double factor = matrix[j][i] / matrix[i][i];
24            for (size_t k = i; k <= n; k++) {
25                matrix[j][k] -= factor * matrix[i][k];
26            }
27        }
28    }
29    if (isSingular) {
30        errorMessage = "The system has no single solution.";
31        isSolutionExists = !hasContradiction(n, matrix);
32        return false;
33    }
34    return true;
35 }

```

Листинг 1 — Прямой шаг метода Гаусса.

```

1 bool hasContradiction(const int n, vector<vector<double>>&
   matrix) {
2     vector<double> zeroes(n, 0);
3     for (size_t i = 0; i < n; i++) {
4         vector<double> curr_row = matrix[i];
5         curr_row.pop_back();
6         if (curr_row == zeroes && matrix[i][n] != 0) {
7             return true;
8         }
9     }
10    return false;
11 }

```

Листинг 2 — Проверка системы на совместность.

```

1 vector<double> solveByGauss(int n, vector<vector<double>> matrix
  ) {
2   vector<double> answer(2 * n, 0);
3   vector<vector<double>> matrix_triangle = matrix;
4   if (!triangulateMatrix(n, matrix_triangle)) {
5     return answer;
6   }
7   for (size_t i = 0; i < n; i++) {
8     const size_t idx = n - i - 1;
9     for (size_t j = idx + 1; j < n; j++) {
10      answer[idx] -= matrix_triangle[idx][j] * answer[j];
11    }
12    answer[idx] += matrix_triangle[idx][n];
13    answer[idx] /= matrix_triangle[idx][idx];
14  }
15  // calculate residuals
16  for (size_t i = 0; i < n; i++) {
17    for (size_t j = 0; j < n; j++) {
18      answer[n + i] += matrix[i][j] * answer[j];
19    }
20    answer[n + i] -= matrix[i][n];
21  }
22  return answer;
23 }

```

Листинг 3 — Обратный шаг метода Гаусса.

2.3.2 Программный интерфейс

Для указанного метода был реализован программный интерфейс, который позволяет переключаться между следующими режимами:

1. Пользовательский ввод.
2. Ввод данных из файла.
3. Генерация случайных данных.

Пользовательский ввод обрабатывает возможные ошибки со стороны пользователя и предлагает исправить их как можно раньше, а не ждать ввода всей матрицы.

Ввод данных из файла поддерживает два вида разделителей, которые используются в вещественных числах: и точку, и запятую.

Генерация случайной матрицы осуществляется с учетом задаваемой пользователем размерности.

Далее приведён листинг исходного кода программы, которая реализует описанный интерфейс.

```

1 #include "lib/preamble.h"
2 #include "lib/gauss.h"
3 #include "lib/input.h"
4 #include "lib/output.h"

```

```

5 #include "lib/random.h"
6
7 int main() {
8     int source = InputModule::getInputSource();
9     vector<vector<double>> augmented;
10    int n = 0;
11
12    if (source == 1) {
13        n = InputModule::getDimension("console");
14        augmented = InputModule::readMatrixFromConsole(n);
15    } else if (source == 2) {
16        string filename = InputModule::getFilename();
17        augmented = InputModule::readMatrixFromFile(filename);
18        if (augmented.empty()) {
19            return 1;
20        }
21        n = augmented.size();
22    } else if (source == 3) {
23        cout << "Enter system dimension (n): ";
24        n = InputModule::getDimension("console");
25
26        cout << "\nGenerating random system..." << endl;
27        vector<vector<double>> A = generateMatrixA(n);
28        vector<double> x_original = generateUnknowns(n);
29        vector<double> b = calculateF(A, x_original);
30
31        augmented = A;
32        for (int i = 0; i < n; i++) {
33            augmented[i].push_back(b[i]);
34        }
35
36        cout << "\nOriginal solution:" << endl;
37        outputVector(x_original.begin(), x_original.end());
38    }
39
40    cout << "\n### SOLVING SYSTEM" << endl;
41    cout << "Given matrix:" << endl;
42    output2DVector(augmented);
43    vector<double> x_solved = solveByGauss(n, augmented);
44
45    if (errorMessage != "") {
46        cout << errorMessage << endl;
47        cout << "Do solutions exist? " << (isSolutionExists ? "Yes"
48            : "No") << endl;
49        return 1;
50    }
51    return 0;
52 }

```

Листинг 4 — Файл main.cpp

```

1 #ifndef PREAMBLE_H

```

```

2 #define PREAMBLE_H
3
4 #include <bits/stdc++.h>
5
6 #include <iostream>
7
8 using namespace std;
9
10 string ltrim(const string&);
11 string rtrim(const string&);
12 vector<string> split(const string&);
13
14 inline bool isSolutionExists = true;
15 inline string errorMessage = "";
16
17 #endif

```

Листинг 5 — Файл preamble.h

```

1 #ifndef OUTPUT_H
2 #define OUTPUT_H
3
4 #include "preamble.h"
5
6 void outputTrace(const vector<vector<double>>& t_matrix, const
    vector<double>& answer);
7 void output2DVector(const vector<vector<double>>& arr);
8
9 template <typename Iterator>
10 void outputVector(Iterator begin, Iterator end) {
11     for (Iterator it = begin; it != end; ++it) {
12         cout << *it << (next(it) == end ? " " : " ");
13     }
14     cout << endl;
15 }
16
17 #endif

```

Листинг 6 — Файл output.h

```

1 #include "output.h"
2
3 void outputTrace(const vector<vector<double>>& t_matrix, const
    vector<double>& answer) {
4     int n = t_matrix.size();
5
6     double det = 1.0;
7     for (int i = 0; i < n; ++i) {
8         det *= t_matrix[i][i];
9     }
10     cout << "1. Determinant: " << det << endl;
11     cout << "2. Triangular matrix:" << endl;

```

```

12  output2DVector(t_matrix);
13  cout << "3. Root:" << endl;
14  outputVector(answer.begin(), next(answer.begin(), n));
15  cout << "4. Residuals:" << endl;
16  outputVector(next(answer.begin(), n), answer.end());
17 }
18
19 void output2DVector(const vector<vector<double>>& arr) {
20     for (int i = 0; i < arr.size(); ++i) {
21         for (int j = 0; j < arr[i].size(); ++j) {
22             cout << setw(10) << fixed << setprecision(4) << arr[i][j]
23                 << " ";
24         }
25         cout << endl;
26     }
27 }

```

Листинг 7 — Файл output.cpp

```

1  #ifndef RANDOM_H
2  #define RANDOM_H
3
4  #include "gauss.h"
5  #include "preamble.h"
6
7  vector<vector<double>> generateMatrixA(int n);
8  vector<double> generateUnknowns(int n);
9  vector<double> calculateF(const vector<vector<double>>& A, const
    vector<double>& x);
10
11 #endif

```

Листинг 8 — Файл random.h

```

1  #include "random.h"
2
3  vector<vector<double>> generateMatrixA(int n) {
4      vector<vector<double>> A(n, vector<double>(n));
5      random_device rd;
6      mt19937 gen(rd());
7      uniform_real_distribution<double> dist(-10.0, 10.0);
8
9      for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             A[i][j] = dist(gen);
12         }
13     }
14
15     for (int i = 0; i < n; i++) {
16         double rowSum = 0;
17         for (int j = 0; j < n; j++) {
18             if (i != j)

```

```

19         rowSum += abs(A[i][j]);
20     }
21     if (abs(A[i][i]) <= rowSum) {
22         A[i][i] = rowSum + abs(A[i][i]) + 1.0;
23     }
24 }
25
26 return A;
27 }
28
29 vector<double> generateUnknowns(int n) {
30     vector<double> x(n);
31     random_device rd;
32     mt19937 gen(rd());
33     uniform_real_distribution<double> dist(-10.0, 10.0);
34
35     for (int i = 0; i < n; i++) {
36         x[i] = dist(gen);
37     }
38
39     return x;
40 }
41
42 vector<double> calculateF(const vector<vector<double>>& A, const
    vector<double>& x) {
43     int n = A.size();
44     vector<double> b(n, 0.0);
45
46     for (int i = 0; i < n; i++) {
47         for (int j = 0; j < n; j++) {
48             b[i] += A[i][j] * x[j];
49         }
50     }
51
52     return b;
53 }

```

Листинг 9 — Файл random.cpp

```

1 #ifndef INPUT_H
2 #define INPUT_H
3
4 #include <cctype>
5 #include <filesystem>
6 #include <fstream>
7 #include <sstream>
8
9 #include "preamble.h"
10 #ifdef _WIN32
11 #include <direct.h>
12 #else
13 #include <unistd.h>

```

```

14 #endif
15
16 namespace fs = std::filesystem;
17
18 class InputModule {
19 private:
20     static bool isDouble(const string& str) {
21         if (str.empty())
22             return false;
23
24         string s = str;
25         size_t start = s.find_first_not_of(" \t");
26         if (start == string::npos) {
27             return false;
28         }
29         size_t end = s.find_last_not_of(" \t");
30         s = s.substr(start, end - start + 1);
31
32         if (s.empty()) {
33             return false;
34         }
35
36         bool decimalPoint = false;
37         bool hasDigit = false;
38         bool signAllowed = true;
39
40         for (size_t i = 0; i < s.length(); i++) {
41             char c = s[i];
42
43             if (c == '+' || c == '-') {
44                 if (!signAllowed)
45                     return false;
46                 if (i + 1 >= s.length())
47                     return false;
48                 signAllowed = false;
49             } else if (c == '.' || c == ',') {
50                 if (decimalPoint) {
51                     return false;
52                 }
53                 decimalPoint = true;
54                 signAllowed = false;
55             } else if (isdigit(c)) {
56                 hasDigit = true;
57                 signAllowed = false;
58             } else if (c == ' ' || c == '\t') {
59                 if (i > 0 && (s[i - 1] == '+' || s[i - 1] == '-'))
60                     return false;
61                 bool hasOnlySpaces = true;
62                 for (size_t j = i; j < s.length(); j++) {
63                     if (s[j] != ' ' && s[j] != '\t') {
64                         hasOnlySpaces = false;

```

```

65         break;
66     }
67 }
68 if (hasOnlySpaces)
69     break;
70 return false;
71 } else {
72     return false;
73 }
74 }
75
76 return hasDigit;
77 }
78
79 static double parseDouble(const string& str) {
80     string s = str;
81     size_t start = s.find_first_not_of(" \t");
82     size_t end = s.find_last_not_of(" \t");
83     s = s.substr(start, end - start + 1);
84
85     size_t commaPos = s.find(',');
86     if (commaPos != string::npos) {
87         s[commaPos] = '.';
88     }
89
90     return stod(s);
91 }
92
93 public:
94 static int getDimension(const string& source = "console") {
95     int n = -1;
96     string input;
97     bool valid = false;
98
99     while (!valid) {
100         cout << "Enter system dimension (n > 0): ";
101
102         if (source == "console") {
103             getline(cin, input);
104         }
105
106         bool isNumber = true;
107         for (char c : input) {
108             if (!isdigit(c) && c != '-' && c != '+') {
109                 isNumber = false;
110                 break;
111             }
112         }
113
114         if (!isNumber || input.empty()) {
115             cerr << "Error: Invalid input! Please enter a positive

```

```

116         integer." << endl;
117     continue;
118 }
119 try {
120     n = stoi(input);
121     if (n <= 0) {
122         cerr << "Error: Dimension must be positive! Please
123             enter n > 0." << endl;
124     } else if (n > 100) {
125         cerr << "Warning: Large dimension may cause
126             performance issues. Continue? (y/n): ";
127         string choice;
128         getline(cin, choice);
129         if (choice == "y" || choice == "Y") {
130             valid = true;
131         }
132     } else {
133         valid = true;
134     }
135 } catch (...) {
136     cerr << "Error: Invalid dimension! Please enter a valid
137         integer." << endl;
138 }
139
140 return n;
141 }
142
143 static vector<vector<double>> readMatrixFromConsole(int n) {
144     vector<vector<double>> A(n, vector<double>(n, 0.0));
145     vector<double> b(n, 0.0);
146
147     cout << "\nEnter matrix A (" << n << "x" << n << "):" <<
148         endl;
149     cout << "Format: " << n << " numbers per row, space-
150         separated" << endl;
151
152     for (int i = 0; i < n; i++) {
153         bool rowValid = false;
154         while (!rowValid) {
155             cout << "Row " << i + 1 << ": ";
156             string line;
157             getline(cin, line);
158
159             vector<string> tokens;
160             stringstream ss(line);
161             string token;
162             while (ss >> token) {
163                 tokens.push_back(token);
164             }
165         }
166     }

```

```

161
162     if (tokens.size() != n) {
163         cerr << "Error: Expected " << n << " numbers, got " <<
164             tokens.size()
165             << ". Please re-enter row " << i + 1 << ":" <<
166             endl;
167         continue;
168     }
169
170     bool rowValidNumbers = true;
171     for (int j = 0; j < n; j++) {
172         if (!isDouble(tokens[j])) {
173             cerr << "Error: Invalid number '" << tokens[j] << "'
174                 at position " << j + 1
175                 << " in row " << i + 1 << endl;
176             rowValidNumbers = false;
177             break;
178         }
179     }
180
181     if (!rowValidNumbers) {
182         continue;
183     }
184
185     for (int j = 0; j < n; j++) {
186         A[i][j] = parseDouble(tokens[j]);
187     }
188     rowValid = true;
189 }
190
191 cout << "\nEnter RHS vector b (" << n << " elements):" <<
192     endl;
193 for (int i = 0; i < n; i++) {
194     bool elementValid = false;
195     while (!elementValid) {
196         cout << "b[" << i + 1 << "]: ";
197         string input;
198         getline(cin, input);
199
200         if (!isDouble(input)) {
201             cerr << "Error: Invalid number! Please enter a valid
202                 number." << endl;
203             continue;
204         }
205
206         b[i] = parseDouble(input);
207         elementValid = true;
208     }
209 }

```

```

207     vector<vector<double>> augmented = A;
208     for (int i = 0; i < n; i++) {
209         augmented[i].push_back(b[i]);
210     }
211
212     return augmented;
213 }
214
215 static vector<vector<double>> readMatrixFromFile(const string&
216     filename) {
217     ifstream file(filename);
218     if (!file.is_open()) {
219         cerr << "Error: Cannot open file '" << filename << "'" <<
220             endl;
221         return {};
222     }
223
224     int n;
225     file >> n;
226
227     if (file.fail() || n <= 0) {
228         cerr << "Error: Invalid dimension in file!" << endl;
229         return {};
230     }
231
232     vector<vector<double>> augmented(n, vector<double>(n + 1,
233         0.0));
234
235     for (int i = 0; i < n; i++) {
236         for (int j = 0; j <= n; j++) {
237             if (!(file >> augmented[i][j])) {
238                 cerr << "Error: Not enough numbers in matrix at row "
239                     << i + 1 << endl;
240                 return {};
241             }
242         }
243     }
244
245     double extra;
246     if (file >> extra) {
247         cerr << "Warning: Extra data found in file after reading
248             all required numbers." << endl;
249     }
250
251     file.close();
252     return augmented;
253 }
254
255 static int getInputSource() {
256     string choice;
257     while (true) {

```

```

253     cout << "### INPUT SOURCE SELECTION" << endl;
254     cout << "1. Console input" << endl;
255     cout << "2. File input" << endl;
256     cout << "3. Generate random system" << endl;
257     cout << "Choose option (1-3): ";
258
259     if (!getline(cin, choice)) {
260         cerr << "\nError: End of input detected. Exiting." <<
                endl;
261         exit(1);
262     }
263
264     if (choice.empty()) {
265         cerr << "Error: No input provided!" << endl;
266         continue;
267     }
268
269     if (choice == "1") {
270         return 1;
271     }
272     if (choice == "2") {
273         return 2;
274     }
275     if (choice == "3") {
276         return 3;
277     }
278
279     cerr << "Error: Invalid choice '" << choice << "'" <<
            endl;
280 }
281 }
282
283 static string getFilename() {
284     string filename;
285
286     while (true) {
287         cout << "Enter filename: ";
288
289         if (!getline(cin, filename)) {
290             cerr << "\nError: End of input detected. Exiting." <<
                    endl;
291             exit(1);
292         }
293
294         if (filename.empty()) {
295             cerr << "Error: Filename cannot be empty!" << endl;
296             continue;
297         }
298
299         if (fs::exists(filename) && fs::is_regular_file(filename))
            {

```

```

300     return filename;
301 }
302
303 fs::path testPath = fs::path("tests") / filename;
304 if (fs::exists(testPath) && fs::is_regular_file(testPath))
305 {
306     cout << "Found file in 'tests/' directory: " << testPath
307         << endl;
308     return testPath.string();
309 }
310
311 cerr << "Error: File '" << filename << "' not found in '
312     tests/' subdirectory!" << endl;
313 }
314 #endif

```

Листинг 10 — Файл input.h

2.4 Результаты работы программы

2.4.1 Внешний ввод данных

Для тестирования программы используем предложенные наборы данных:

1. Нулевые матрицы разных размерностей с ненулевым вектором f :

$$A_1 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}_{5 \times 5}, \quad f_1 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$$

$$A_2 = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{pmatrix}_{10 \times 10}, \quad f_2 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_3 = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{pmatrix}_{20 \times 20}, \quad f_3 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 20 \end{pmatrix}$$

2. Матрицы со всеми элементами единицами разных размерностей, f также ненулевой:

$$A_4 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}_{5 \times 5}, \quad f_4 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$$

$$A_5 = \begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}_{10 \times 10}, \quad f_5 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_6 = \begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}_{20 \times 20}, \quad f_6 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 20 \end{pmatrix}$$

3. Матрицы разных размерностей, у которых легко рассчитать определитель:

$$A_7 = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 10 \end{pmatrix}_{5 \times 5}, \quad f_7 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 0 \\ 5 \end{pmatrix}$$

$$A_8 = \begin{pmatrix} 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 \end{pmatrix}_{10 \times 10}, \quad f_8 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_9 = \begin{pmatrix} 5 & 1 & 0 & \cdots & 0 \\ 1 & 5 & 1 & \ddots & \vdots \\ 0 & 1 & 5 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & 5 \end{pmatrix}_{20 \times 20}, \quad f_9 = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

Проверим вывод для каждого набора данных:

Матрица	Результат
A_1	Ошибка: система не имеет решений
A_2	Ошибка: система не имеет решений
A_3	Ошибка: система не имеет решений
A_4	Ошибка: система не имеет решений
A_5	Ошибка: система не имеет решений
A_6	Ошибка: система не имеет решений

См. далее

Матрица	Результат
A_7	Ошибка: система имеет бесконечно много решений
A_8	Нулевая невязка
A_9	Нулевая невязка

Таблица 1: Результаты тестирования

2.4.2 Случайные матрицы

Для тестирования программы на случайных данных сгенерируем пять примеров расширенных матриц разных размерностей. Укажем входные данные в их исходном формате.

Расширенная матрица пяти переменных:

```
5 40.4564 -9.9887 -8.8804 -9.3251 -1.5994 336.8412 -5.2606 34.5
663 7.4763 7.9874 -4.3633 -8.6072 -1.5834 -9.9567 26.8659 3.049
0 -3.4104 -293.8578 7.7408 -7.5478 -5.4741 29.9140 1.6049 -212.
2229 3.5968 6.2100 7.9053 7.6499 29.2512 180.3360
```

Решение системы:

x_1	6.4461	x_2	5.2916	x_3	-6.5044	x_4	-9.0671	x_5	8.3782
-------	--------	-------	--------	-------	---------	-------	---------	-------	--------

Результирующий вектор невязки:

r_1	0.0000	r_2	-0.0000	r_3	0.0000	r_4	0.0000	r_5	-0.0000
-------	--------	-------	---------	-------	--------	-------	--------	-------	---------

Расширенная матрица семи переменных:

```
7 40.2033 2.8908 7.4733 -5.4225 -7.9681 -2.7764 -3.6029 -292.33
45 9.0126 30.5033 0.4669 5.8591 -5.2665 0.8368 1.8789 265.8340
-4.1489 -2.0737 28.6430 -3.2608 -8.6993 2.6820 -4.4365 32.4776
0.4299 -5.6963 7.1359 33.3470 -6.3016 -3.3665 -1.3772 199.7044
-4.2855 2.8858 -1.0049 2.0486 25.2903 3.0650 -6.3273 93.4829 -2
.5839 9.6242 -2.5045 5.7705 -4.2655 43.6585 8.8450 -144.5704 0.
2904 -3.9326 -7.8012 -2.0600 -3.2564 2.4359 28.8232 -9.3104
```

Решение системы:

x_1	-7.3746	x_2	9.9399	x_3	3.5723	x_4	6.9389
x_5	2.6198	x_6	-7.0975	x_7	3.4660		

Результирующий вектор невязки:

r_1	0.0000	r_2	0.0000	r_3	0.0000	r_4	-0.0000
r_5	0.0000	r_6	-0.0000	r_7	0.0000		

Расширенная матрица десяти переменных:

10 69.2954 -8.1930 8.1595 0.4206 -8.9363 -9.7974 -1.3126 9.4670
-6.9505 -9.7259 169.0100 -1.5225 49.2972 -7.2712 -3.6430 5.6608
8.6770 4.6706 -1.9809 -3.4781 9.3148 -15.7867 6.6136 -4.1672 35
.5724 -4.9478 -5.9128 3.1665 -1.9357 0.3204 -4.3461 0.6519 -141
.4718 8.6516 -9.9743 5.1889 65.6150 -6.9734 8.6404 4.9645 -9.89
95 -5.3320 4.3482 568.2128 7.8031 -9.5426 3.8111 -1.0234 56.364
5 -6.1708 -8.6405 -2.6918 7.3792 2.6469 -364.6524 9.1998 -8.784
1 -4.6314 2.2858 1.6782 44.8025 3.2826 -1.2128 0.8249 -5.6682 3
32.4950 -2.4676 -9.9431 -4.7010 3.1623 -0.3566 -8.7803 58.1439
-6.4740 8.6134 7.5314 -395.8601 4.6407 1.4052 8.6245 4.8565 -6.
4513 -6.5800 -0.6544 41.4927 -2.9164 -3.5292 106.0667 -2.6502 2
.6281 -3.4279 0.6428 -3.8632 0.4669 6.8241 7.0707 33.5404 -1.98
61 -48.8460 -4.0958 4.2049 -9.6623 5.0959 -7.7835 2.4755 9.7390
4.2995 9.0160 66.0224 -257.8359

Решение системы:

x_1	1.8981	x_2	0.5323	x_3	-5.1110	x_4	8.3044	x_5	-5.6498
x_6	6.2359	x_7	-5.3769	x_8	1.8408	x_9	-2.3640	x_{10}	-5.1142

Результирующий вектор невязки:

r_1	-0.0000	r_2	0.0000	r_3	0.0000	r_4	0.0000	r_5	0.0000
r_6	0.0000	r_7	0.0000	r_8	-0.0000	r_9	0.0000	r_{10}	-0.0000

Расширенная матрица шестнадцати переменных:

16 82.1898 0.8281 -6.9379 7.6740 6.3111 3.2602 2.0987 -2.0539 4
.5068 -4.0142 9.5368 5.8353 7.5398 3.7165 -8.0881 -0.1342 -72.7
010 -3.3287 63.6497 -4.4173 -6.1183 0.9080 4.7982 -1.2634 5.949
4 1.5641 0.2751 -0.4314 -1.5864 8.6999 5.6316 3.1670 9.9275 97.
6764 -0.9463 -2.2945 70.2341 6.7532 6.8806 -1.3839 -2.3382 7.60
80 6.4766 -7.6348 -3.2973 0.0176 -7.1779 -5.2122 -5.6294 5.126
4 242.0010 -5.1615 -8.2589 4.7951 96.5448 9.7892 -9.3486 -6.054
9 -7.0683 9.6358 -7.5451 -9.7002 -3.8167 -2.3999 -2.4077 -6.188
8 -1.4108 -910.0916 -6.7380 -4.6775 -3.9086 -5.4457 93.3653 -2.
2377 -8.9563 9.7573 8.9672 -9.2755 -5.1208 -4.5674 3.6832 5.138
8 -6.7079 0.2784 903.8162 7.6288 -1.2814 2.6302 -7.5930 4.0429
82.0850 -6.9025 3.6373 9.7095 0.8828 5.8091 -3.0205 -1.8785 -8.
6762 2.0762 8.2284 632.7161 -8.5796 6.2456 7.6270 8.5995 7.6205
-6.5963 106.6256 -8.0482 3.4630 8.4009 9.9323 -1.1303 5.9525 -6
.4671 -9.1231 3.9099 -272.5719 -3.3073 6.2986 5.1119 -8.4819 0.
1970 6.6351 -1.2994 73.2348 -4.5043 1.7696 6.8869 5.5690 8.2524
-3.8161 -2.2599 2.9229 234.6717 -6.2258 0.5344 -4.2923 -9.1852
-8.2930 -7.8747 -2.4929 3.3836 92.1754 7.2962 9.4512 -5.3782 7.
6497 4.5501 -9.8322 -1.9423 -608.0678 1.5242 4.7579 -6.8674 1.4
079 2.9466 2.2629 1.2604 -4.3922 2.6289 70.2055 -9.7256 -0.0837
1.9316 -8.7573 5.8298 -5.6463 -538.6969 1.4046 -4.7280 -9.8171

-2.6420 1.6958 8.8411 7.5039 7.0547 7.3869 7.6244 93.7094 -4.05
81 -5.3906 -8.9069 7.9114 -2.1676 419.6901 0.3146 -6.5603 6.245
7 -4.0665 -0.3398 7.8353 -7.6930 7.3568 -7.7221 4.1928 2.2104 7
5.7502 -2.4023 1.2372 -0.0611 8.2636 304.5823 -3.3609 -7.0454 -
8.2763 7.4829 8.2197 -5.3143 -0.0384 9.1381 -7.7885 -3.3528 5.9
834 8.2142 89.0893 -0.7621 -2.8495 5.1888 -552.3446 8.2044 -8.8
410 3.9604 -0.0264 -9.6070 1.9527 6.3772 -4.4969 8.0335 1.4630
0.7497 -5.8511 -5.9049 78.6010 4.1172 -6.6069 -251.2824 -2.5582
7.8265 6.7708 9.6327 -7.6576 6.7071 5.8238 -1.9322 -9.6783 5.28
85 8.3316 4.3464 8.0464 -9.5879 108.6138 3.9854 -847.6057 1.936
0 3.4685 2.3854 4.3091 3.1085 1.9593 -7.4163 5.2879 1.3482 -3.6
029 0.6773 -7.9746 9.4231 -5.7073 7.9354 74.3561 -545.1952

Решение системы:

x_1	-1.4796	x_5	9.3101	x_9	-6.2632	x_{13}	-6.7313
x_2	2.5748	x_6	7.2760	x_{10}	-6.8811	x_{14}	-1.4661
x_3	2.6911	x_7	-2.1736	x_{11}	4.9961	x_{15}	-7.0971
x_4	-9.8026	x_8	0.6979	x_{12}	2.5878	x_{16}	-6.2693

Результирующий вектор невязки:

r_1	0.0000	r_5	0.0000	r_9	0.0000	r_{13}	0.0000
r_2	-0.0000	r_6	0.0000	r_{10}	0.0000	r_{14}	0.0000
r_3	0.0000	r_7	-0.0000	r_{11}	-0.0000	r_{15}	-0.0000
r_4	-0.0000	r_8	0.0000	r_{12}	-0.0000	r_{16}	0.0000

Расширенная матрица двадцати переменных:

20 113.8302 -5.6305 -8.0658 4.5681 1.0307 -7.8755 -6.5325 -7.82
19 3.3604 -9.2476 6.7013 -4.2878 7.1581 -2.4441 -6.6217 -3.4428
7.6514 -1.6179 -8.2688 3.2434 -744.9563 -8.8384 118.5891 -3.605
9 -6.9450 -9.0348 -0.7075 9.5702 -8.4072 0.8933 2.5789 -7.6380
4.6716 -9.5750 -2.5941 6.0315 9.9047 4.7401 7.8765 3.9453 1.680
5 -84.6917 6.1427 1.5096 104.4844 -7.4113 2.7249 7.4613 -3.5819
-6.2203 7.1019 -4.3935 8.8569 7.1445 -4.1530 -1.8966 2.8698 7.8
682 8.8713 -1.6218 -3.6874 -1.7594 -631.8203 -4.5672 2.9681 0.1
322 126.6744 9.6932 4.0034 -8.4168 5.9274 -4.4390 -7.9609 8.267
1 7.3430 -7.5077 4.5866 -6.8529 6.9085 -5.6079 -5.5887 9.0266 8
.3470 -1147.5783 -0.2159 8.9800 3.3487 -5.5697 102.8247 -5.1157
2.8759 -8.3268 4.5777 3.8227 8.8132 7.0325 -9.4962 -7.7379 0.61
30 3.6263 -3.1009 -6.7497 4.0569 -5.8380 259.4853 -2.2408 -2.39
86 -8.1605 -6.4704 1.1176 89.4389 -1.0065 2.1301 8.5256 -2.1364
-3.8771 -9.5115 -2.8753 9.7645 3.1152 -6.5124 -8.3767 2.0347 -0
.2165 -7.7702 116.3441 -6.4849 -8.3173 6.9353 -5.1831 9.0828 6.
4158 104.8762 -2.0639 -6.9444 5.0334 0.9246 -7.4967 -2.9374 4.4
316 0.8528 -3.2860 -7.3838 -8.8076 -2.7234 2.3870 -137.4070 -2.
0168 -4.1500 -4.5768 -0.5555 3.5155 -3.2060 7.5190 79.5720 -7.9
661 4.7874 -5.1107 -1.7521 4.5330 -2.0943 6.9906 -3.9213 -4.009

3 -0.8951 7.6942 0.8711 133.8926 2.2724 -2.4279 -3.9434 7.7845
-7.4786 -5.5461 2.9212 0.8571 79.8797 2.0876 -3.6917 -5.9545 -0
.2028 5.5348 -2.3611 7.5240 6.7981 -2.8708 -7.2115 1.0162 119.3
634 0.4924 1.6513 7.0691 -3.7633 6.9286 3.7200 7.7637 -3.4843 2
.1420 90.1497 8.2935 4.6198 -0.0559 1.4275 9.7138 -4.7132 -3.0
639 -2.2100 1.6292 7.7383 506.1682 3.7986 7.8787 0.7540 -9.174
6 7.7445 1.4639 0.2527 4.0814 8.8810 9.6360 94.2860 -1.8095 2.6
913 7.1874 -2.6225 -4.8380 -0.6895 -4.5665 7.3503 -0.5095 -855.
1237 -0.0102 -1.9511 -0.1574 3.4697 -4.7632 8.9619 7.0205 -5.89
09 -3.3991 3.9142 7.2903 93.0605 -6.7593 -4.7059 1.9433 -6.3458
-7.1113 1.7040 -7.2743 -7.9621 -717.2087 7.0032 7.4792 6.6845 8
.3489 -0.6610 5.9718 3.4923 4.0402 2.9680 -0.9378 9.2400 -2.287
7 106.2905 -4.6162 9.1533 -2.0970 -7.9947 7.8208 0.5488 8.9468
-349.6369 5.5251 8.4851 5.9745 -4.3589 -7.7291 -7.0949 7.7265 -
8.9318 4.1903 5.9397 7.3851 -7.5357 -9.2591 122.3489 -5.9600 0.
2087 6.9592 -3.3521 3.2937 7.9612 -115.7796 5.3215 3.6217 -1.96
25 -4.0438 0.4640 -4.2715 2.1326 -2.0931 -1.6396 5.6718 -8.6915
-4.4388 8.5162 1.5423 83.4476 6.0788 8.2939 4.2528 1.6316 2.765
1 -14.0596 -5.9549 5.9982 6.3205 -1.3554 -3.8465 -0.3499 4.9161
9.6815 -9.8357 -8.0103 7.9013 2.6077 -9.4781 -6.0829 -1.2772 11
8.4453 -7.8891 -1.6237 -5.5487 -9.1347 731.8699 1.4070 5.8482 -
3.9900 -3.6382 1.4876 3.2112 -6.5146 8.5418 -9.6582 -3.9540 8.4
374 6.7198 0.7756 -4.7722 -5.8603 -9.1854 98.7106 -4.6547 -0.80
92 -3.6166 -99.7080 -1.9070 9.7978 9.3286 8.9379 3.2336 -8.4964
-3.8899 5.8129 6.0184 -1.4366 2.6613 -4.9837 8.6221 3.6188 -8.9
703 -5.6192 3.3711 107.0222 5.1205 0.3757 497.0815 -9.4492 -9.8
027 0.8232 -1.6980 7.6326 9.0727 3.3562 -8.8185 -5.1508 8.7883
-8.8759 -5.6916 0.5870 9.8939 0.6923 6.9177 -0.4799 -6.9920 121
.3280 -6.6253 -358.0187 2.1106 -6.5386 3.0657 8.8495 4.5482 -5.
8717 1.3629 -6.8542 3.5457 7.7321 -9.1966 -7.6033 6.0723 -0.760
0 8.8463 1.5011 -3.6184 -4.5376 -5.6524 108.0962 841.8304

Решение системы:

x_1	-6.2885	x_6	0.1656	x_{11}	-9.5156	x_{16}	7.9173
x_2	-2.8401	x_7	-2.4370	x_{12}	-5.9057	x_{17}	0.9769
x_3	-5.5308	x_8	1.1420	x_{13}	-1.4088	x_{18}	6.2717
x_4	-8.7554	x_9	0.3857	x_{14}	-0.6205	x_{19}	-4.9910
x_5	4.1633	x_{10}	7.2734	x_{15}	-2.8878	x_{20}	7.0276

Результирующий вектор невязки:

r_1	-0.0000	r_6	0.0000	r_{11}	0.0000	r_{16}	-0.0000
r_2	0.0000	r_7	0.0000	r_{12}	0.0000	r_{17}	0.0000
r_3	0.0000	r_8	0.0000	r_{13}	-0.0000	r_{18}	-0.0000
r_4	0.0000	r_9	-0.0000	r_{14}	-0.0000	r_{19}	-0.0000
r_5	-0.0000	r_{10}	0.0000	r_{15}	0.0000	r_{20}	-0.0000

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

Почему у матриц $A_1, \dots, A_7$ возникла ошибка? Матрицы $A_1, \dots, A_3$ обладают общим свойством: их определитель равен нулю. Метод Гаусса не может использоваться для вырожденных матриц, поскольку спроектирован для поиска единственного решения. В нашем случае решений нет: для любых векторов система противоречива из-за ненулевого $\vec{f}$.

Матрицы $A_4, \dots, A_6$ внешне ничем не примечательны, но можно заметить, что их строки линейно зависимы. Это тоже означает, что решение не может быть единственным. После приведения расширенных матриц к верхнетреугольному виду выяснится, что в системах также есть противоречие, и решений нет.

Наконец, матрица A_7 имеет нулевой элемент на главной диагонали. Более того, на этой же строке имеется нулевой элемент у $\vec{f}$. Это значит, что x_4 — свободная переменная, и решений у системы бесконечно много.

Какая асимптотическая сложность у алгоритма? Временная сложность метода Гаусса составляет $\mathcal{O}(n^3)$. Как следует из блок-схемы, прямой ход содержит три вложенных цикла (*по переменным i, j, k*). Обратный ход содержит два вложенных цикла, потому имеет сложность $\mathcal{O}(n^2)$. Пространственную сложность можно оценить $\mathcal{O}(n^2)$, поскольку алгоритм требует хранения всей матрицы.

Почему бы не воспользоваться методом Крамера? По сравнению с обычным методом Крамера, метод Гаусса эффективнее для систем большой размерности. Метод Крамера требует вычисления $n + 1$ определителя, что является асимптотически сложной вычислительной операцией, а конкретно — $\mathcal{O}(n^4)$.

Почему невязка везде нулевая? Предположительно это связано с видом рассматриваемых матриц. Как можно заметить, в отчёте приведены матрицы с относительно небольшими по модулю коэффициентами и малой дробной частью (*либо вообще целые числа*).

Если бы была дана матрица с коэффициентами, у которых мантисса велика, то точность вычисляемых значений упала бы из-за ограниченности разрядной сетки типа `double`. Для нашей реализации метода это было бы особенно заметно, потому что все операции последовательные, и ошибка бы только накапливалась, что привело бы к ненулевой невязке.